

Efficient VLA for RaccoonBot Control Plane

2024404060 강민혁

June 14, 2026

요약

본 보고서는 mujoco 시뮬레이션 환경에서 OpenVLA-7B를 통해 4 DOF manipulator RaccoonBot 로봇을 제어하도록 하는 파이프라인 제시한다. 우선, 객체를 Pick-and-Place하는 태스크와 이에 대응하는 새로운 자연어 명령을 추가하고, 이를 기존 grasp 데이터와 통합해 단일 LoRA 기반의 멀티태스크 학습을 수행하였다. 이와 함께 4 DOF manipulator 로봇에 매핑하는 과정을 데이터 수집부터 실물 제어 전 단계에서 공통으로 사용하도록 일원화했다. 시스템 전반에 걸쳐 속도 및 메모리 최적화를 적용하고 진단 metric을 구축하여 observability를 확보하였다. 특히 단일 RTX 5080 (16GB) 환경에서 OpenVLA-7B의 학습과 추론을 동시에 수행할 수 있도록 asymmetric mixed-precision quantization을 설계 및 구현하였으며, 구축된 추론 서버에 실물 로봇 클라이언트를 연동하여 실제 제어 검증까지 완료하였다. 결과적으로 asymmetric quantization을 통해 최대 학습 메모리를 약 14.1GB로 절감하여 16GB 단일 GPU에서의 학습을 성공적으로 수행하였다. 약 5000 스텝의 학습 과정에서 과적합 없이 train loss는 약 0.27을 달성하였고, accuracy는 약 0.9가 나옴을 확인하였다. 하지만, 실제 mujoco closed-loop 평가에서는 시뮬레이션과 실물 모두에서 grasp 성공률이 기대에 미치지 못하는 결과가 나왔다. 정성적 추측이 아닌 정량적 계측을 기반으로 closed-loop 제어 실패 원인을 규명하였다. 본 과제를 통해 Openvla를 통해 실제 7D-to-4DOF 액션 생성에 대한 파이프라인을 설계하였고, 실제 로봇에 적용하는 귀중한 경험을 할 수 있었다.

초기에는, 여러 개 쌓인 실린더에서 젠가와 같이 중간 특정 실린더만 빼내는 작업을 목표로 하여 진행하였다. 하지만, 제공된 A100 GPU의 VRAM이 늘 가용 불가할 정도로 높은 점유율을 보였으며, SM 및 대역폭도 항상 높은 유지되어 제대로 학습을 시킬 수 없었다. 새벽에 확인하여도, 낮에 확인하여도, 저녁에 확인하여도 늘 자원이 부족하였으며, 특정 프로세스가 항상 자원을 독점하고 있음을 확인하였다. 운 좋게 학습을 돌릴 수 있어도, step당 speed가 3.2로 로컬에서 돌리는 것과 유사한 속도가 나왔으며, 다른 프로세스가 함께 돌기 시작하니 5.3까지 느려졌다. 따라서, 기존 생각했던 태스크 대신, 최적화 쪽으로 진행을 하였다.

1 시스템 개요 및 아키텍처

1.1 시스템 개요

본 과제에서 사용한 로봇은 RaccoonBot으로, 4 DOF manipulator와 gripper를 갖추고 있다. mujoco 시뮬레이션 환경의 모델과 실물 RaccoonBot은 동일한 kinematics를 공유한다(6장). 모델로는 OpenVLA-7B를 사용한다. DINOv2와 SigLIP을 결합한 이중 비전 백본, Llama-2 7B 언어모델, 그리고 256구간 행동 토큰라이저로 구성되며, LoRA로 파인튜닝 가능하다. 데이터셋 생성 및 시뮬레이션 환경은 mujoco이고 관측은 카메라 RGB 영상이다. 실제 환경에서는 노트북의 웹 캠을 사용한다.

1.2 전체 흐름

우선, mujoco를 활용해 pick-and-place task에 대한 데모를 병렬로 생성한다. 수집된 데이터는 로봇 train data 형식으로 변환 및 구축되며, 이를 바탕으로 OpenVLA-7B 모델에 대한 LoRA 파인튜닝을

수행한다. 학습이 완료된 모델은 시뮬레이션 평가와 실물 제어 환경 모두에 inference를 진행한다. 이 모든 과정은 단일 진입점(2장)으로 실행 가능하며, 실행 내역은 모두 시스템 로그로 기록되도록 하였다.

모델이 7D action space [dx, dy, dz, droll, dpitch, dyaw, gripper] 상의 명령을 출력하면, 클라이언트는 사전에 정의된 공통 행동 매핑 로직(4.1절)을 적용하여 이를 4 DOF 로봇 제어를 위한 IK로 해석한다. 보상은 sparse reward로 진행하였다. 특히, 추론 서버는 요청을 발생시키는 클라이언트에 의존하지 않도록 독립적으로 설계하였다.

2 파이프라인 인프라

2.1 설계 개요

각 실행 단계를 독립적인 스크립트로 분리하되 번호를 부여하여 관리할 수 있도록 하였다. 이때, OpenVLA 코드베이스 전체를 수정하는 것은 한계가 있기에 대부분은 기존 코드를 하위 subprocess로 호출하는 wrapper 형태로 작성했다. 이때, 전체 파이프라인은 단일 진입점으로 제어되도록 하였다. 또한, 분산 학습이나 mujoco 환경에서 zombie 혹은 orphan process를 방지하기 위해, 정상 종료나 시그널 발생 시 파생된 모든 하위 프로세스까지 깔끔하게 정리하도록 하였다.

2.2 argument 설정 및 실행 기록

각 단계의 진행 여부와 argument는 config 파일에 작성하였으며, 적용 우선순위는 명령행 인자, 설정 파일, 코드 기본값 순이다.

매 실행마다 Git 커밋 해시, 단계별 종료 코드 및 소요 시간, 선택된 단계, 그리고 최종 상태를 JSON manifest 형태로 기록하였다. 실행이 실패하거나 중간에 중단되더라도 해당 시점까지의 부분 기록이 보존되며, 콘솔 창에는 단계별 소요 시간이 표시된다. 파이프라인을 구성하는 세부 단계는 표 1와 같다.

표 1. 단계형 파이프라인 구성

단계	동작	GPU
환경 점검	경로, 라이브러리, 패키지 등 확인	—
데모 생성	집기·쌓기 데모 생성	(CPU 병렬처리)
데이터 변환	raw 영상 데이터를 변환 후 학습 데이터셋 빌드	—
에피소드 시각화	프레임 몽타주 이미지 생성	—
학습	샌드위치 QLoRA	사용
병합·추론	LoRA 사후 병합 및 FP8 추론 서버 구동	사용
오프라인 검증	예측값과 정답 간의 거리 비교 분석	사용
closed-loop 평가	시뮬레이션 환경 집기·쌓기 성공률 검증	사용
실제 제어	추론 서버와 클라이언트를 통한 RaccoonBot 구동 (6장)	사용

3 데이터셋 확장

3.1 확장 내용

기존 파이프라인은 대상 객체가 원기둥에 국한되고, 수행 가능한 작업이 단순한 '집기'뿐이며, 명령어 역시 "원기둥을 집어라" 단 하나로 고정되어 있다는 한계가 존재했다. 본 과제에서는 정육면체

객체와 '집어서 쌓기' task를 도입하고 이에 대응하는 명령어를 추가하였다. 물론, 앞에서 서술한 바와 같이 더욱 복잡하고 재미있는 task를 진행하고자 하였지만, 자원의 한계로 진행하지 못하였다.

3.2 쌓기 성공 판정

성공 판정은 기존 예제의 grasp용 접촉 기준을 그대로 쓰지 않고 새로 정의했다. 실린더의 평면 위치가 베이스 위치의 허용 오차 이내이고, 높이가 베이스 상단에 실린더 절반 높이를 더한 값에 허용 오차 이내로 맞으며, 실린더가 베이스와 접촉하고, 그리퍼가 열려 있으며, 움직임이 충분히 느려 안정된 상태를 성공으로 본다. 쌓기는 본질적으로 해제 후의 안착이 중요하기에, 접촉만으로 판정하면 적절하지 않기 때문이다.

4 코드 개선

4.1 7D-to-4DOF

7D-to-4DOF RaccoonBot에 대응시키는 매핑 로직을 하나로 통합하여, 데이터 수집, 학습 데이터 생성, closed-loop 시뮬레이션 평가, 그리고 RaccoonBot 제어 등 모든 단계에서 동일하게 재사용할 수 있도록 구조를 개선하였다.

또한, step별 위치 변위의 axis별 제한값을 argument로 설정할 수 있도록 하여, 학습 데이터 분포의 상위 분위수에 맞춰 유연하게 조정할 수 있게 했다. 매핑 함수는 초기 변위, 최종 적용 변위, 목표 위치, gripper 명령, IK 재시도, 실패 여부 등 다양한 진단 정보를 반환하며, 평가 단계에서는 이를 로깅 및 분석에 활용한다(4.3절).

특히, end-effector의 자세를 도출하는 kinematics 연산은 grasp task에서 큰 오차가 발생함을 확인하여 데이터 수집 단계와 시뮬레이션 환경 단계가 동일한 kinematics 상수를 공유하여 수행되도록 통일했다. RaccoonBot 제어 또한 동일한 매핑을 사용하여, 시뮬레이션에서 검증된 동작을 실제 로봇이 재현할 수 있도록 하였다(6장).

4.2 추론·모션 속도 및 메모리 최적화

학습, 추론, 모션 제어, 데이터 생성에 이르는 파이프라인 전 구간에 걸쳐 연산 속도 향상 및 메모리 최적화 기법을 도입했다. 각 구간별로 적용된 구체적인 기법과 그에 따른 성능 개선 효과를 표 2에 요약했다.

개선 전후 비교는 학습 로그의 최대 할당·예약 메모리와 스텝 시간으로 정량화, inference latency는 평가 리포트의 평균 inference latency와 실물 제어 시 서버 로그의 타임스탬프 간격으로 측정한다(6장).

4.3 타이밍 및 행동 로깅과 시각화

본 파이프라인은 실험의 재현성을 보장하고 제어 실패 원인을 정밀하게 분석하기 위해 로깅 및 시각화 체계를 구축하여 시스템 observability을 확보하였다. 우선, 시스템 실행 기록(2.2절)을 통해 Git 커밋 해시, 모듈 단계별 종료 코드 및 소요 시간을 자동으로 기록하여 실험 간 성능 비교를 위한 기반 데이터를 제공한다. 이와 더불어 Closed-loop 평가 과정에서는 매 스텝마다 clipping 발생률, gripper 폐쇄 유지 스텝 수 IK 해 탐색 실패 및 재시도 횟수, end-effector의 누적 이동 거리 등의 세부 제어 지표를 수집하여 평가 리포트로 산출한다. 이러한 정량적 계측 데이터는 제어 실패 원인이 부적절한 action scale 출력에 있는지, gripper grasp 타이밍의 문제인지, 혹은 단순 IK 연산 한계인지를 객관적으로 판별하는 데 활용된다(8장).

표 2. 구간별 최적화 기법 및 효과

기법	구간	효과
Flash-Attention 2	학습	attention buffer를 제거하여 활성화 메모리를 줄이고, kernel fusion으로 연산 속도 향상.
메모리 fragmentation 완화 할당자	학습	메모리 fragmentation로 인해 낭비되는 메모리 공간을 최소화.
8bit optimization	학습	LoRA optimization state를 32bit에서 8bit으로 quantization.
LoRA 사후 병합	학습	매 체크포인트마다 14GB 규모의 weight를 병합하는 대신, 학습 종료 후 1회만 병합, 전체 throughput을 극대화
물리 연산 반복 축소	데모 생성	physics pass 횟수를 100회에서 50회로 축소하여 step당 연산 비용을 절감. 50까지 줄였음에도 anti-slip에 문제가 없음을 확인하였다.
병렬 rollout 워커	평가	멀티코어 기반으로 병렬로 rollout을 수행하며, 모든 worker가 단일 GPU 기반 policy server를 통해 평가 효율 향상.
FP8 추론	추론	Blackwell tensor core의 FP8 가속 및 deterministic decoding 활용 RaccoonBot 제어 주기 latency 최소화

또한, 결과 분석의 직관성을 높이기 위해, 학습 데이터 검증에 위한 에피소드 프레임 몽타주, 평가 지표 집계 그래프, 추론 과정에서 로봇의 동작을 프레임 단위로 보여주는 rollout 프레임 몽타주를 생성한다. 이에 더해, ClearML을 바탕으로 실험 추적을 진행하여 train loss부터, accuracy, L1 loss 등을 체계적으로 관리할 수 있도록 하였다.

5 추론 및 평가

오프라인 검증은 데모 프레임 한 장에 대한 예측과 정답의 거리만을 확인한다. 학습이 실제로 이루어졌는지를 사람이 눈으로 가늠하는 정도이다. 다만, 이로는 실제 로봇을 작동시킬 때, 오차가 크다는 것을 확인하여 정책을 mujoco에서 여러 차례 rollout하여 실제 과제 성공률(수집과 동일한 성공 기준을 재사용), 성공까지의 스텝 수, 평균 추론 지연을 자동으로 산출하는 closed-loop 평가를 진행하였다. 베이스 모델과 파인튜닝 모델의 성공률 차이로 학습 효과를 정량화하였다.

서버는 명령 텍스트와 카메라 이미지를 받아 7차원 행동을 돌려주는, state가 없는 HTTP 예측 엔드포인트일 뿐이며, 누가 요청하든 동일하게 응답한다.

추론과 mujoco 소프트웨어 렌더링을 한 프로세스에 올리면 충돌이 발생함을 확인하였으며, 완전히 뜯어고치지 않는 한 구조적으로 해결이 불가능함을 확인하였다. 렌더링 worker를 별도 프로세스로 분리하고, wrapper가 서버 동작부터 워커 실행, 서버 종료를 순서대로 진행한다. 이는 충돌 회피를 넘어 decoupling도 된다. 같은 서버에 시뮬레이션 평가 워커 대신 실물 클라이언트를 연결하면 곧바로 실물 제어도 가능하다(6장).

6 RaccoonBot 제어

RaccoonBot 제어는 RaccoonBot이 RaccoonBot 및 카메라와 연결된 클라이언트가 추론 서버와 통신하는 방식으로 하여 RaccoonBot을 구동하였다.

6.1 배포 구조

수업 때 받은 GPU 서버는 포트포워딩 불가 문제로 외부에서 직접 접근할 수 없어, FP8 추론 서버는 개인 데스크탑에 띄웠다. 클라이언트는 개인 랩탑을 사용하였으며, 랩탑에 RaccoonBot을

연결하고, 랩탑의 웹 캠을 사용하여 비디오를 추론 서버로 전송하였다. 서버에 예측을 요청해서 7D action을 받은 뒤, 이를 바탕으로 RaccoonBot에 명령을 내린다. 서버는 본 과제에서 개선한 FP8 추론 서버로, 7B 정책을 단일 16GB VRAM에 올리고(7장) latency를 최소화 하여 real-time 제어를 가능하게 하였다.

6.2 시뮬레이션 검증의 실물 이식성

본 과제에서는 데이터 수집, 학습 라벨링, 시뮬레이션 평가, 실물 적용이라는 네 단계 전반에 걸쳐 동일한 7D-to-4DOF(4.1절의 동일한 축별 클립, 동일한 IK, 동일한 kinematics 상수)를 적용하였다. 이를 closed-loop 시뮬레이션 환경에서 측정한 성공률과 실패 양상이 실제 구동 시의 신뢰성을 예상할 수 있도록 하였다.

7 VRAM 절감과 비대칭 Quantization

수업에서 접속권한을 받은 A100 서버의 가용 자원이 일관되지 않아 개인 데스크탑(RTX 5080(Blackwell, 16GB))으로 학습 및 추론을 진행하였다. 이때, OpenVLA-7B의 BF16 전체 LoRA 학습은 16GB에서 메모리 부족으로 실패하였다. 이를 해결하기 위해 모듈별 비대칭 mixed-precision 샌드위치 구조의 QLoRA를 구현했다.

7.1 모듈별 비대칭 Quantization

VLA는 모듈마다 quantization 민감도가 크게 다르기에, 모든 모듈에 같은 quantization을 적용하는 방식은 피해야 한다. quantization은 한 곳으로 모아 학습과 추론이 공유한다. 모듈별 학습 및 추론 precision은 표 3와 같다.

표 3. Asymmetric Mixed-Precision

모듈	학습	추론
ViT(DINOv2 + SigLIP)	BF16 + LoRA ($r = 16$)	INT8 Post-training Quantization
프로젝터(MLP)	BF16 전체 미세조정	BF16 유지
LLM backbone(Llama-2 7B)	FP8 freeze + LoRA ($r = 32$)	FP8 (W8A8)
Action Output Layer	BF16 전체 fine-tune	BF16 유지

OpenVLA는 수업에서 배웠던 것과 같이 로봇 제어를 위해 독립적인 action head 모듈을 두지 않고, 언어 모델의 최종 출력 logits 중 256개의 action vocabulary에서 최댓값을 얻고, 이를 denormalization해서 연속적인 제어 값으로 변환하는 방식을 사용한다. 만약, 시스템 경량화를 위해 언어 모델의 백본 전체를 FP8로 Quantization할 경우 최종 출력 logit의 미세한 오차가 누적되어 제어에 실패할 위험이 있다. 따라서 본 프로젝트에서는 양자화 범위를 언어 모델 본체로만 제한하고, action logit을 연산하는 출력 층은 quantization하지 않도록 하였다.

7.2 구현

OpenVLA 아키텍처가 특정 transformers 버전을 요구하기에 모델 가중치를 우선 메모리에 올리고, 이후 독립적인 모듈로 quantization을 하였다. 샌드위치 QLoRA 구성에서는 언어 모델 백본을 FP8로 quantization 및 freeze(약 7.5GB)한 상태에서 어텐션의 Q, V projection층에 LoRA($r = 32$)를 부착하였고, vision encoder는 BF16에서 LoRA($r = 16$)를 적용하였다. vision-language projector와 action head는 full fine-tuning 대상으로 하여 BF16으로 학습을 진행하였다. 학습 효율성을 높

이기 위해 gradient checkpointing, 8-bit optimizer, flash-attention 2으로, batch size 4와 gradient accumulation steps 4를 설정하여 유효 배치 크기 16을 달성할 수 있었다.

추론 단계에서는 현재 vLLM 프레임워크가 prismatic 기반 이중 백본을 지원하지 않아, 기존 FastAPI 기반의 추론 서버를 구현하였다. 이때, OOM을 방지하기 위해, CPU 메모리에 모델을 먼저 로드한 후 layer-wise하게 quantization하여 GPU로 넘기도록 하였다. 이를 통해 약 14.5GB의 VRAM만으로 모델을 올릴 수 있었다.

8 결과

8.1 학습

샌드위치 QLoRA로 학습할 때 최대 메모리는 약 14.1GB(할당 피크 약 14.1GB, 예약 피크 약 14.3GB)로, 16GB를 가진 RTX 5080 한 장에서 충분히 학습 가능하였다.

약 5,000 스텝 학습에서 학습 손실은 초기 약 8.7에서 수백 스텝 안에 0.5로 줄어든 뒤, 5,000 스텝까지 점진적으로 줄어들다 최종적으로 validation loss 0.3 수준에서 수렴하였다. L1 오차는 거의 0에 붙어, 과적합 징후 없이 일반화되었음을 보인다. accuracy는 약 0.55에서 시작하여 수백 스텝 안에 약 0.76을 달성하였으며, 이르고, 이후 약 0.88에서 정체하며, 검증 정확도도 같은 수준으로 함께 오른다. FP8 동결 백본에 BF16 LoRA를 더한 샌드위치 구성이 수렴과 일반화를 해치지 않음을 학습 곡선으로 확인하였다. 학습 속도는 약 스텝 당 3.7초(RTX 5080, 유효 배치 16)이다.

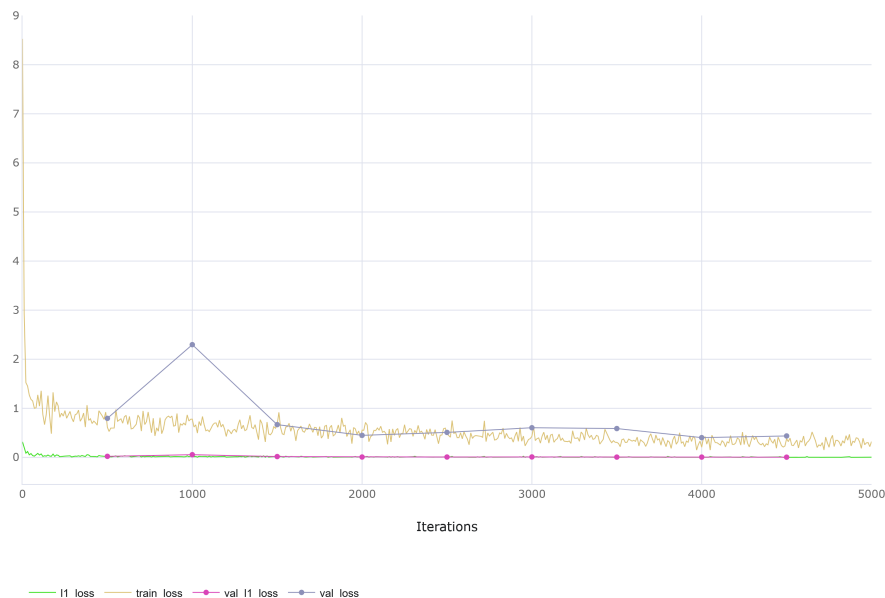


그림 1. 학습·검증 손실 곡선. 가로축은 학습 스텝(0 ~5,000)이다. 학습 손실이 약 8.7에서 급강하한 뒤 약 0.3으로 수렴하고, 검증 손실이 이를 동반하며 L1 오차는 거의 0에 붙는다.

8.2 시뮬레이션 평가

쌍기 closed-loop 평가는 전 에피소드가 실패했고, 그 원인을 여러 단계에 걸쳐 측정하며 좁혀 나갔다. 단계별 가설과 조치, 판정은 표 4와 같다.

최종 085 closed-loop 평가, 제어 주기 10Hz, 에피소드당 최대 321 스텝의 대표 에피소드는 표 5와

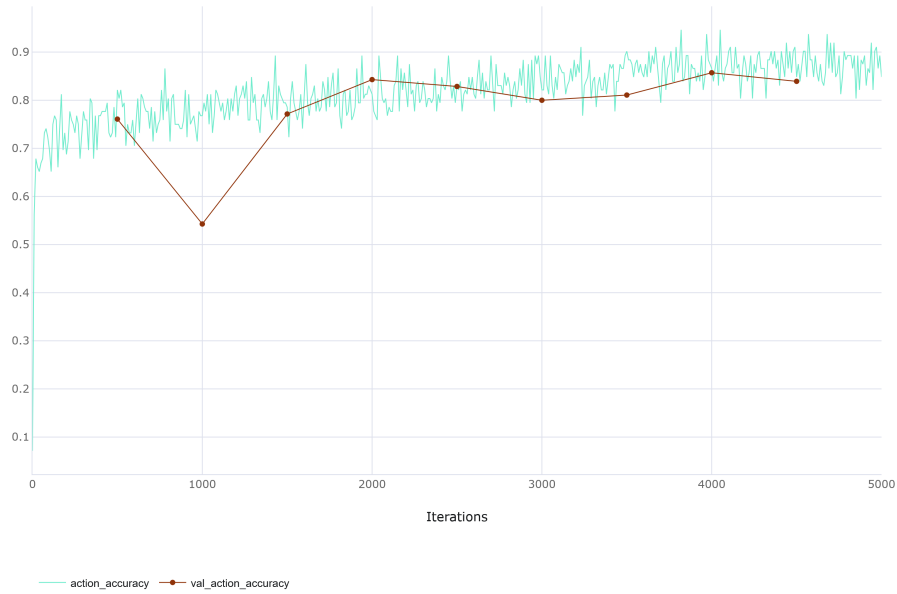


그림 2. 행동 정확도 곡선. 가로축은 학습 스텝(0 ~5,000)이다. 0.65에서 출발해 1~2천 스텝에 약 0.8, 이후 약 0.88에서 정체하며 학습과 검증이 함께 움직인다.

표 4. 쌓기 closed-loop 실패 파악

단계	원인 가설	조치	판정
1차	행동 라벨 붕괴(0·상수 토큰 지배)와 kinematics 프레임 불일치	정지 프레임 제거, kinematics 재생성	부분 해결(증상 잔존)
2차	학습 변위(수 cm)와 rollout 클립(1 cm)의 3~7배 불일치	제어 주기 세분화로 변위 축소	부분 해결(라벨 희석 부작용)
3차	평가 주기와 생성 주기의 5배 불일치, 최대 스텝 부족, 측정 부재	주기 동기화, 최대 스텝 증가, 스텝별 측정 추가	정합 확보
4차	재생성·재학습 후 그리퍼·클립·주기·IK·kinematics를 모두 배제	—	집기 정밀도로 확정

같으며, 모두 스텝별 측정에 근거하였다. FP8 추론 서버의 평균 추론 지연은 약 179ms으로, 단일 16GB GPU에서 실시간 제어가 가능하였다.

특정 색상의 실린더로 이동하여 도달하는 과정, gripper 작동, clipping, IK 및 kinematics은 모두 정상적으로 동작함을 확인하였다. 따라서 근본적인 실패 원인은 grasp 위치와 타이밍의 정밀도 부족인 것으로 추정된다. 즉, 모델이 정확한 위치와 시점에 실린더를 잡지 못하는 정밀도 문제로 분석하였다. 이러한 문제는 클리핑이나 제어 주기 등의 하이퍼파라미터 설정만으로는 해결하기 어려우며, 훈련 데이터의 다양성과 양, 그리고 전반적인 학습량을 늘려 해결해야 한다.

grasp 실패 후 로봇 팔이 위쪽이나 바깥쪽으로 벗어나는 현상은 부차적인 증상으로, Out-of-Distribution 문제에 해당한다. 스크립트 기반의 데모는 모두 성공적인 궤적으로만 구성되어 있기 때문에 실패한 상태 공간은 학습 분포에 포함되지 않는다. grasp 원인이 해결되면 이 현상도 해결될 것으로 예상된다.

grasp 정밀도가 실패의 주요 원인이라는 것은 rollout에서 grasp 순간을 측정한 결과에 직접 확인할 수 있다(figure 3). 이 세 가지 rollout에서 모델이 gripper를 닿는 순간의 XY 평면 오차는 16.6 cm에서 0.93 cm까지 다양하게 분포한다. 이때, XY 정렬 오차가 grasp 허용 오차에 사실상 도달할 정도로 정확하게 접근한 경우조차 실린더의 이동량이 0에 머문다는 것이다. 노란색 rollout의 경우 gripper가 실린더 바로 위 0.68 cm 지점까지 정확하게 접근했음에도 실린더를 들어 올리지 못했다.

표 5. 대표 에피소드 실제 측정 기반 확인

항목	값	해석
성공 여부	실패	—
gripper 폐쇄 스텝 수	281	gripper는 닫힘을 확인. gripper 문제 배제
클립 발생률	0.0	변위가 잘리지 않음을 확인. 클립 문제 배제)
IK 실패·재시도 스텝 수	0	IK 정상
엔드이펙터 이동 거리	0.95 m	팔은 충분히 동작
최초 폐쇄 시 XY 거리	0.166 m	실린더에서 16.6cm 떨어진 채 gripper를 닫음
최소 XY 접근 거리	0.096 m	한 번도 집기 허용오차(0.75 cm)에 근접하지 못함
실린더 이동량	0.0 m	실린더를 들어올리지 못함. 집기 실패 확정.

색상 인식이나 XY 평면상의 접근 능력은 충분하지만, 하강 높이, gripper 시점 등이 복합적으로 작용하는 동작 자체가 제대로 학습되지 않았음을 예상할 수 있다.

figure 4는 이러한 수평 및 수직 오차를 2차원 평면에 함께 시각화한 것으로, gripper를 닫는 순간의 pose를 XY 평면 거리와 end-effector의 Z축 높이로 나타내면, 이상적인 grasp 자세(XY=0, Z 2 cm)를 중심으로 하는 grasp 영역에 어떤 rollout도 도달하지 못했다. 빨간색 rollout은 XY 좌표에서 크게 벗어나 있으며, 가장 정렬이 잘 된 노란색 rollout조차 해당 경계선 밖에 머물러 있다. 결과적으로 모델은 올바른 grasp 자세에서 미세하지만 일관되게 빗나가고 있다. 실제로 정렬 상태가 가장 우수한 노란색 rollout의 연속 프레임(figure 5)을 살펴보면, 로봇 팔이 실린더가 모여 있는 위치까지 하강하기는 하지만 결국 어떤 실린더도 제대로 grasp하지 못 한 것을 확인할 수 있다.

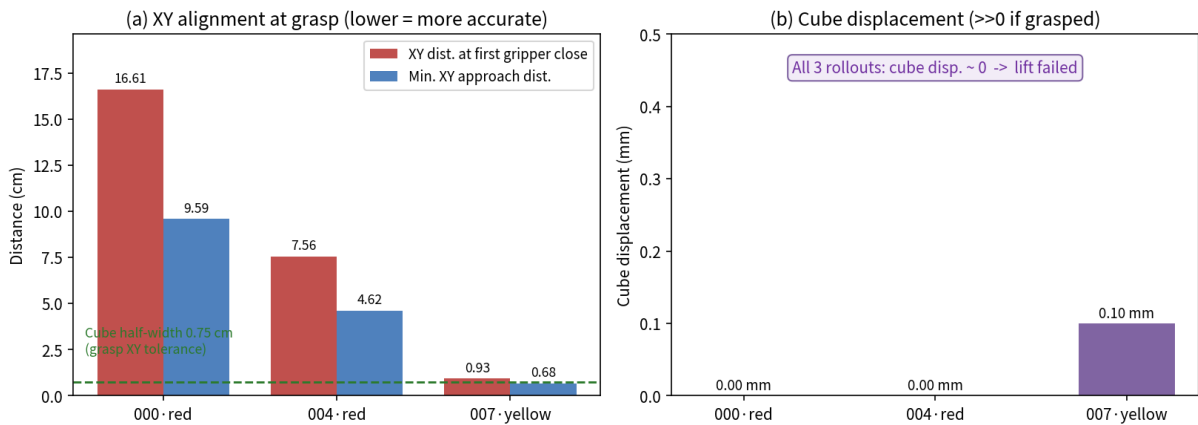


그림 3. grasp 정밀도 진단(대표 rollout 3건). (a) 그리퍼를 닫는 순간의 XY 정렬과 최소 XY 접근 거리. 노랑 rollout은 grasp 허용오차(점선, 실린더 반폭 0.75 cm) 이내까지 정렬한다. (b) 그림에도 세 rollout 모두 실린더 이동량이 0에 수렴하여 들어올림에 실패한다. 평면 접근은 가능하나 grasp가 실패함을 알 수 있다.

8.3 RaccoonBot 제어

학습한 결과를 바탕으로 추론서버를 통해 RaccoonBot을 구동하였으나, 성공하지 못하였다. end-effector는 매 스텝 위쪽으로만 향하는 동작을 반복하였고, gripper는 닫혔다 열리기를 간헐적으로 반복할 뿐 어떤 실린더에도 도달하거나 grasp하지 못하였다(figure 6).

추론 서버는 dx, dy, dz만을 생성하고, droll, dpitch, dyaw는 항상 0만을 반환하여 end-effector의 방향이 변경되지 않았다.

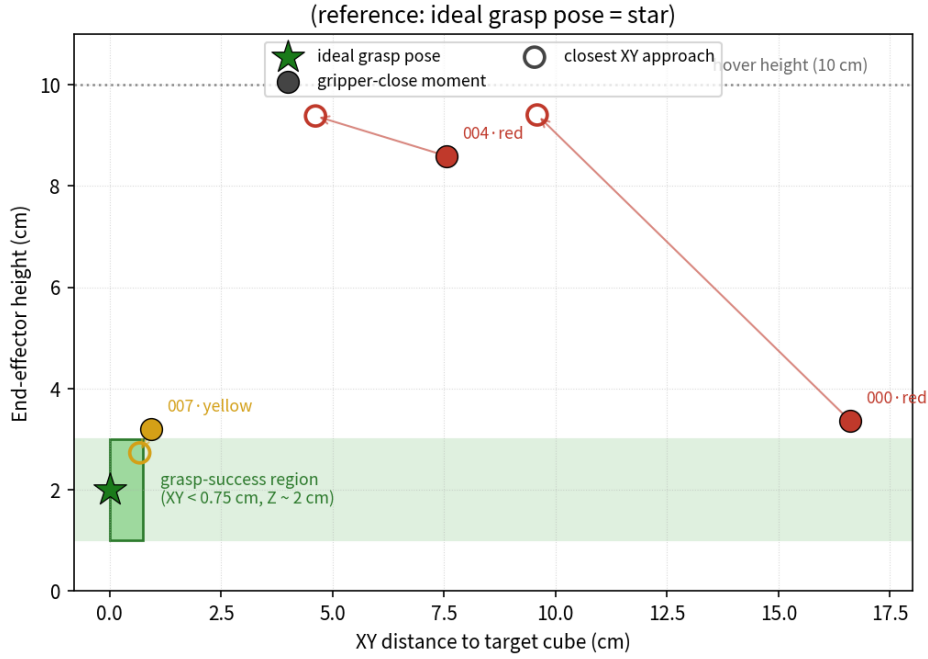


그림 4. grasp 순간 pose 오차(대표 rollout 3건). 가로축은 대상 실린더까지의 XY 거리, 세로축은 end-effector 높이이다. 별표는 이상적 grasp 자세(XY = 0, Z ≈ 2 cm), 녹색 영역은 grasp 영역(XY < 0.75 cm, Z ≈ 2 cm)이다. 채워진 표식은 gripper를 닫는 순간, 빈 표식은 최소 XY 접근 순간이다. 빨강 계열은 XY에서 크게 벗어나고, 가장 잘 정렬된 노랑조차 grasp 영역의 경계 바로 위에 머물러, 어떤 rollout도 지점에 접근하지 못하였다.

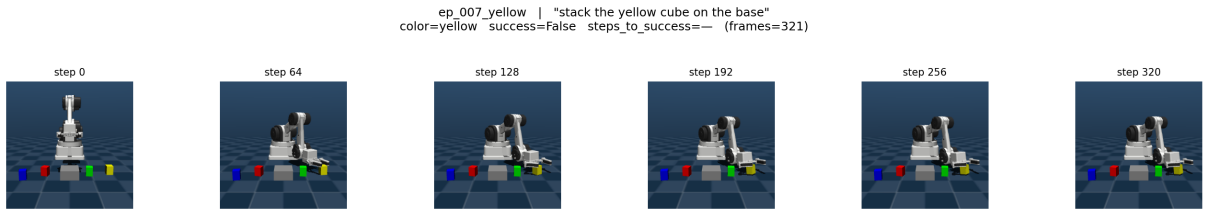


그림 5. XY 정렬이 가장 정확했던 rollout(노랑, stack the yellow cube on the base)의 프레임. 팔이 실린더까지 정확히 하강하지만 실린더를 grasp하지 못하였다.

9 한계 및 future works

본 과제에서 가장 큰 문제점은 grasp의 정밀도 부족이다. 8.2절에서 확인했듯이, 모델은 색상 인식과 XY 평면 이동을 통해 목표 실린더 바로 위까지는 안정적으로 도달한다. 하지만 하강 높이와 gripper 개폐 시점이 정교하게 맞물려야 하는 실제 grasp 동작은 일관되게 수행하지 못했다. 이는 특정 하이퍼파라미터 설정이나 구현 세부 사항의 문제라기보다는, 성공 데모만으로 정밀한 접촉 동작을 학습해야 하는 형태의 구조적 한계로 해석된다. 이러한 현상은 시뮬레이션과 실제 환경 양쪽에서 공통으로 관찰되었으며, 따라서 시뮬레이션 상의 성능 개선을 동반하지 않은 단순한 값 전송만으로는 해결하기 어렵다.

8.3절에서 확인할 수 있듯이, RaccoonBot 검증에서는 이와는 별개의 문제점도 확인되었다. 추론 서버가 이동 좌표(dx, dy, dz)만 생성하고 회전 성분(droll, dpitch, dyaw)은 항상 0으로 반환하여, end-effector의 방향이 전혀 갱신되지 않는 현상이 발생했다. 그 결과 매 스텝 로봇 팔이 위쪽으로 고정된 채 어떤 실린더에도 닿지 못했다. 이는 학습된 모델 자체의 결함이라기보다는 행동 표현과 서버 추론 파이프라인 간의 정합성 문제이므로, 성능을 본격적으로 평가하기 전에 반드시 수정해야 할 구현상의 문제이다.

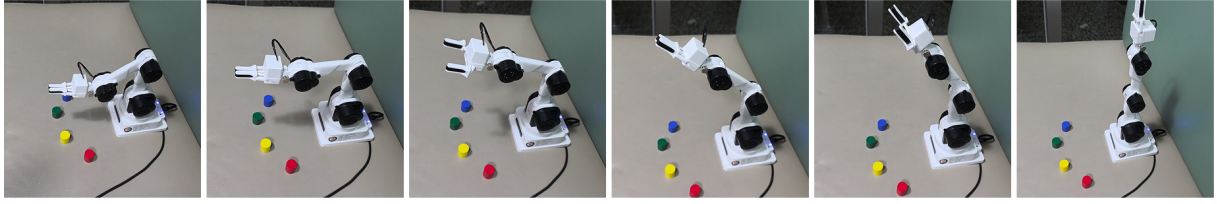


그림 6. RaccoonBot 제어 시도의 동영상 일부. end-effector가 매 스텝 위쪽으로만 향한 채 고정되어 실린더에 도달하지 못한다. 추론 서버가 자세 성분(droll, dpitch, dyaw)을 반환하지 않아 방향이 갱신되지 않으며, gripper만 간헐적으로 여닫힌다.

방법론적 측면에서는 hyperparameter tune을 완전히 자동화하지 못한 점이 아쉬움으로 남는다. P 기반 전이학습과 ASHA 알고리즘을 활용한 자동 탐색을 검토하였으나, 고정된 너비의 파인튜닝 환경, 단일 GPU라는 연산 제약, 그리고 validation loss 감소가 실제 closed-loop 성공률 향상으로 직결되지 않는다는 점 때문에 본 OpenVLA 과제 환경에는 부적합하다고 판단했다. 이에 rsLoRA와 제한적인 수동 탐색을 대안으로 적용했으며, 결과적으로 현재의 설정이 최적의 조건이라고 단언하기는 어렵다.

또한, 환경적인 측면에서, 16GB라는 작은 VRAM과 A100환경을 사용하기 위해 낭비한 시간으로 인해 제대로 된 NAS를 진행하지 못하였으며, 동시에 5,000스텝밖에 학습시키지 못하여 제대로 fit시키지 못한 체크포인트를 사용했다는 것이 아쉽게 느껴진다.

이러한 한계를 해결하기 위해, 우선 grasp 정밀도를 높이기 위해 훈련 데이터의 규모와 다양성을 대폭 확보하고, 성공과 실패 사례가 혼합된 데모 생성 바탕으로 negative에 대한 경우에도 정상적으로 작동할 수 있도록 할 필요가 있다. 또한, closed-loop 성공률을 정확히 반영하는 검증 지표를 찾고, 이를 바탕으로 하이퍼파라미터 자동 탐색을 재도입하는 것 또한 의미가 있을 것으로 판단된다.

Acknowledgement

본 과제의 코드 구현에는 Claude-5-fable-(xHigh) 및 Claude-4.8-Opus-(max) 모델을 활용하였습니다. 또한, 보고서 작성에는 Cursor의 인라인 자동완성 기능을 활용하였습니다.